

Лекція 2.1

Тема: Основи мови JavaScript

План.

1. *Впровадження JavaScript у HTML.*
2. *Синтаксис JavaScript.*
 - *Лексема JavaScript.*
 - *Спец. Символи.*
 - *Операції. Змінні.*
 - *Типи Даних.*

Зміст лекції

1. Впровадження сценаріїв в HTML-документи

Одна з незвичайних сторін JavaScript пов'язана з тим, що на цій мові можна створювати програми, що працюють як з боку сервера, так і з боку клієнта, тобто в середовищі браузера. З клієнтської сторони JavaScript застосовується у відповідність з одним з двох способів: впровадження в HTML-документ між парою дескрипторів `<script>` або приміщення всередину дескриптора для забезпечення реакції на подію. Кожен раз коли HTML-сторінка завантажується, JavaScript інтерпретується браузером. Залежно від дій користувача та інших подій, що можуть виникнути під час перегляду HTML-документа, виконується сценарій цілком або його частини.

Наприклад, розробляється HTML-документ для збору даних про відвідувачів певної Web-сторінки. Почати варто з форми, що містить три текстових елемента: ім'я, назва компанії і номер телефону. Кінець форми можна помістити стандартну кнопку Submit (Надіслати) для відправки інформації в базу даних, розміщену на Web-сервері. Крім того, можна зажадати, щоб користувач повністю заповнював кожне поле, а разом з номером телефону вказував код міста.

Після клацання на кнопці Submit всі дані форми передаються на обробку в серверну частину програми. Якщо виявляється, що користувач опустив будь-яку інформацію, видається запит на введення цих даних. Потім користувач може відправити форму, що включає необхідні дані, до того ж в коректному форматі. Після того як програма отримає від користувача легкотравні дані, вона зберігає їх в базі даних.

Підтримка полів, обов'язкових для заповнення, як в даному випадку, призводить до суттєвих затримок, яких можна уникнути за допомогою JavaScript. Має сенс організувати перевірку всіх даних на клієнтській частині - цим не повинен займатися Web-сервер. Реалізація перевірки даних - всього лише один з прикладів того, як JavaScript може доповнювати серверні програми, забезпечуючи швидшу реакцію на помилки і при цьому звільняючи більшу кількість ресурсів на сервері.

Події

Одна з ключових характеристик JavaScript - можливість захоплювати обмежену кількість призначених для користувача дій, в більшості випадків відомих під ім'ям подій (events). Деякі HTML-елементи вже реагують на події; прикладом може послужити клацання на добре відомому елементі посилання, що призводить до перенаправлення на інший HTML-документ.

Як тільки покажчик миші потрапляє на відповідний посиланням текст або графічне зображення, покажчик змінює свій стандартний вид стрілки на вигляд указуючої руки. На додаток до цього, деякі браузери відображають в рядку стану новий URL. В JavaScript подібного роду дія носить назву події `MouseOver`, і реакція буде мати місце тоді, коли виникає будь-якого роду рух миші поверх елемента. Після клацання на будь-якій частині посилання браузер забезпечує перехід на відповідний Web-адреса або відкриття нового файлу. Описане дію називається подією `Click`, яке буде генеруватися в результаті будь-якого клацання на посиланні. HTML перехоплює ці події, і точно такий же буде реакція браузера.

За допомогою JavaScript можна забезпечити призначені для користувача реакції на багато подій, які генеруються під час перегляду HTML-документа користувачем.

Синтаксис

Як і в будь-якій мові програмування, в JavaScript визначено власні способи управління різними елементами. Зрозуміло, в мові є своя специфіка подання ключових слів, зарезервованих слів, символів і т.п.

2. Синтаксис JavaScript

Лексеми (Tokens) - це найменші окремі слова, фрази, символи, які може розпізнати JavaScript. Під час інтерпретації JavaScript-коду браузер розбиває сценарії на лексеми, ігноруючи при цьому коментарі і зайві прогалини.

Лексеми поділяються на чотири категорії: ідентифікатори, ключові слова, літерали і оператори. Як і в будь-якому комп'ютерному мовою, існує безліч способів подачі комп'ютера різних директив за рахунок різної компоновки лексем. Синтаксис (syntax) мови - це набір правил і обмежень, що використовується при комбінуванні лексем.

Ідентифікатори (Identifiers) - суть не що інше як імена змінних, методів та об'єктів. Вони складаються з комбінації букв і цифр. Деякі імена вже вбудовані в мову JavaScript і тому зарезервовані від використання.

Незалежно від ключових слів, можна визначати власні, притому значущі ідентифікатори. Зрозуміло, для цього існує кілька правил:

- Усі ідентифікатори повинні починатися або з літери, або з символу підкреслення (_).
- Для всіх наступних знаків допускається використання букв, цифр і символів

підкреслення.

- У поняття букв входять великі літери, починаючи з A і закінчуючи Z, і малі літери, тобто з a до z.

- Ім'я ідентифікатора не повинно містити прогалин.

У поняття цифр входять знаки, починаючи з 0 і закінчуючи 9.

У табл. 5.6 показані приклади допустимих і неприпустимих ідентифікаторів.

Таблиця 2.1. Приклади ідентифікаторів JavaScript.

допустимі	неприпустимі
current WebSite	current WebSite
numberOfHits	#ofIslands
n	2b0rNotToBe
N	return

Ідентифікатор current WebSite є неприпустимим через присутність в ньому символу пробілу. Цей ідентифікатор буде розглядатися не як один, але як два. Якщо за змістом ідентифікатор повинен перебувати пробіл, то на його місці зазвичай ставиться символ підкреслення.

#ofIslands не є допустимим, оскільки символ # не входить в дозволена безліч символів. 2beOrNotToBe - некоректний, тому як починається з цифри. Ідентифікатор return вже використовується в JavaScript для інших цілей. У разі його визначення, після запуску програми видається помилка.

Ідентифікатори n і N - цілком припустимі, проте відмінні один від одного. З огляду на те що JavaScript чутливий до регістру, згадані ідентифікатори інтерпретуються як різні, навіть незважаючи на їх, в общем-то, однакове вимову.

Ключові слова (Keywords) - попередньо визначені ідентифікатори, що становлять ядро мови програмування. В JavaScript вони виконують особливі функції, наприклад, оголошення нових змінних, функцій, прийняття рішень на основі поточного стану комп'ютера або організація циклу всередині програми.

Ключові слова вбудовані в JavaScript і завжди доступні програмісту, проте відносно них слід дотримуватися правильного синтаксису. Ключове слово var - одне з перших, про який буде

детально розповідатиметься в розділі. Пізніше демонструється застосування інших ключових слів для створення більш динамічних програм.

abstract	else	instanceof	switch
boolean	enum	int	synchronized
break	export	interface	this
byte	extends	long	throw
case	false	native	throws
catch	final	new	transient
char	finally	null	true
class	float	package	try
const	for	private	typeof
continue	function	protected	var
debugger	goto	public	void
default	if	return	volatile
delete	implements	short	while
do	import	static	with
double	in	super	

Літерали (Literals) - це числа або рядки, що представляють постійні значення JavaScript. Це значення, які не змінюються під час виконання сценарію. У наступних п'яти розділах розглядаються різні види літералів.

Цілочисельні літерали можуть бути представлені в десятковому (десятьова система числення), вісімковому (восьмерична система числення) або шістнадцятиричному (шістнадцятковій системі числення) форматі. Цілочисельний літерал в десятковому форматі може містити будь-яку послідовність чисел, але не починатися з 0. Нуль на початку цілочисельного літерала визначає восьмеричний формат.

У вісімковій запису цілочисельний літерал може включати послідовність цифр від 0 до 7. Для визначення шістнадцятиричного формату числа перед літералом має ставитися 0x або 0X. Шістнадцяткові цілочисельні літерали можуть містити цифри від 0 до 9 і букви a - F ілі A - F. Розглянемо кілька прикладів:

Десяткові літерали: 2139
Вісімкові літерали: 03664
Шістнадцяткові літерали: 0x768, 0x395

Літерали з плаваючою точкою представляють десяткові числа з дробовою частиною. Їх можна висловлювати в стандартному і експоненційному форматах. Експонентний формат має на увазі використання e або E для визначення порядку числа. І десятиричне число, і показник ступеня можуть бути зі знаком або беззнаковими (позитивні і негативні):

3405.673
-1.958
8.3200e + 11
8.3200e11
9.98E-12

Логічні літерали

В JavaScript реалізований логічний тип даних, тому підтримуються два літерали true і false, що представляють відповідно значення логічної 1 і 0. Якщо ви новачок в області програмування, не впадайте у відчай - дуже скоро ви зрозумієте всю важливість true і false. Ключові слова true і false повинні записуватися малими літерами. Таким чином, TRUE і FALSE прописними буквами залишаються доступними для застосування в якості власних ідентифікаторів. Однак, щоб уникнути непорозумінь, використовувати їх в такому контексті не рекомендується.

Рядкові літерали

Строковий літерал - це нуль і більше символів, взятих в подвійні (") або поодинокі (') лапки. Незважаючи на те що JavaScript дозволяє використовувати обидва типи лапок, для рядка краще застосовувати один і той же тип лапок як на початку, так і в кінці. Розглянемо приклади строкових літералів:

"Allen's car"

'Virtual "communities"'

"# 12-6"

"Look, up in the sky!"

Застосування одного з двох видів лапок досить зручно, якщо ви - прихильник або першого, або другого методу. Під час вивчення вбудованих методів JavaScript слід врахувати необхідність суворого виконання всіх вказівок, що стосуються строкових літералів, що передаються в якості параметрів. У ряді випадків допускається застосовувати обидва типи лапок при приміщенні одного строкового літерала всередину іншого. Останнє не дозволяється при вказівці керуючих кодів, мова про які йде нижче.

Спеціальні символи

Іноді в сценарії може знадобитися використовувати спеціальний знак або їх комбінацію, наприклад, символи табуляції або нового рядка. У цьому випадку керуючий код повинен передувати символом похилій риси вліво (\), як показано нижче:

\b	- забій
\f	- переклад сторінки
\n	- новий рядок
\r	- повернення каретки
\t	- табуляція
\\	- похила риса вліво
'	- одинарна лапка
"	- лапки

Якщо потрібно симулювати натискання клавіші табуляції для вирівнювання, скажімо, двох стовпців даних, слід використовувати знак табуляції (\ t). У лістингу 1.5 демонструється один із способів вирівнювання тексту за допомогою знаків табуляції. Якщо сценарій містить спеціальні знаки, читати його виявляється складніше, але як показано на рис. 5.1, результати виглядають набагато краще:

Лістинг 2.1. Використання спеціальних знаків в JavaScript.

```
<!DOCTYPE html>
<html>
<head>
  <title> Спеціальні символи в JavaScript-сценарії </title>
</head>
<body>
  <!--
    Зауваження: Спеціальні символи не матимуть дії до тих пір,
    поки не помістити їх в попередньо сформований блок тексту
    (тобто той, який знаходиться між парю дескрипторів <pre> </pre>).-->
  <pre>
    <script type="text/javascript">
      document.writeln("\tПерсонал");
      document.writeln("Ім'я \t \tАдрес");
      document.writeln("Jeff \t \tjjeff@company.com");
      document.writeln("bill \t \tbill@company.com");
      document.writeln("kim \t \tkim@company.com");
    </script>
  </pre>
```

```
</body>
</html>
```

Персонал	
Ім'я	Адрес
Jeff	jeff@company.com
bill	bill@company.com
kim	kim@company.com

МАЛЮНОК 2.1. Вирівнювання тексту за допомогою табуляції

ПРИМІТКА

Спеціальні знаки впливають тільки коли використовуються в межах попередньо відформатованого текстового блоку, тому-то сценарій повинен поміщатися в пару дескрипторів `<pre>` і `</pre>`.

Якщо строковий літерал повинен відтворювати символи лапок, застосовуйте їх в парі з символом похилій риси вліво.

Оператори (Operators) - це символи або ідентифікатори, які представляють спосіб оцінки або маніпуляції комбінаціями виразів. Найбільш використовуваною загальною операцією є далеко не операція присвоювання. У прикладі $x = 10$ як власне 10, так і змінна x - це вираження. Коли JavaScript зустрічає операцію присвоювання між двома виразами, виконуються дії у відповідність з правилами оператора присвоювання. В даному випадку значення виразу в правій частині присвоюється змінній в лівій частині.

Змінні (Variable) - це ім'я, що привласнюється комірці пам'яті комп'ютера, яка зберігає певні дані. Предки програмування витрачали багато часу на трансляцію даних, на зразок повідомлення "Hello World", в двійковій коді. Потім вони шукали вільне місце в комп'ютерній пам'яті для розміщення всіх отриманих одиниць налякав, не забуваючи запам'ятати адресу початку і завершення цих даних. За допомогою згаданих адрес елементів пам'яті можна було знаходити, змінювати і читувати дані. Загалом, будь-коли завершується день бабака. (Хай живе День Сурка !!!)

Нинішнім програмістам пощастило більше - їм на допомогу прийшли змінні, і життя їх пішла на істотну поправку, оскільки збереження, зміну і читання даних стало тепер як ніколи простим. Завдяки механізму змінних з'являється можливість привласнювати значущі імена осередкам, які зберігають дані, явно використовуються програмістом, в той час як доступ костальної осередкам - турбота виключно комп'ютера.

Проблема іменування

Ім'я змінної в JavaScript складається з однієї і більше літер (A..Z і a..z), цифри (0..9) і символів підкреслення (`_`). проте не може починатися з цифри. Наступних два імені JavaScript сприймає як різні:

```
internetAddress
internetaddress
```

Ось ще коректні імена:

```
_lastName
number_2
```

Коли ім'я змінної за змістом містить одне слово, при її записи прийнято використовувати малі літери, а ось при наявності в імені смислових двох або трьох слів, то перше слово починається рядковою, а всі наступні - з прописних. Зазвичай мої імена змінних містять в собі два-три слова, що забезпечує набагато краще розуміння сенсу визначення змінної.

Незважаючи на те що розмір імені змінної JavaScript обмежується тільки об'ємом оперативної пам'яті, все ж підходити до даного питання краще з практичної точки зору. Я радив використовувати не більше 20 символів, тобто 2-3 слова. При написанні сценарію не варто

заходити за кінець рядка. Цьому сприяють довгі імена змінних, до того ж вони порушують цілісність сприйняття коду.

Оголошення змінних

Для того щоб повідомити JavaScript про те, що конкретний ідентифікатор повинен використовуватися в якості змінної, в першу чергу, його потрібно оголосити. Для оголошення змінних в JavaScript використовується ключове слово `var`, за яким слід нове ім'я змінної. Ім'я буде резервуватися як змінна, яку можна використовувати в якості області пам'яті для зберігання будь-яких даних. Наступний приклад демонструє можливість одночасного оголошення кількох змінних (між окремими іменами повинні ставитися коми).

```
var internetAddress;  
var n;  
var i, j, k;  
var isMouseOverLink, helloMessage
```

Після оголошення змінної можна привласнювати початкове значення. Подібна дія носить назву ініціалізації і виконується за допомогою операцією присвоювання `=`.

Змінну можна форматовувати безпосередньо при її оголошенні або пізніше, в будь-якому місці сценарію. Встановлення початкового значення змінної під час оголошення спрощує запам'ятовування, який тип значень спочатку планувалося зберігати в змінної. Уточнимо попередній приклад, додавши необхідні ініціалізації:

```
var internetAddress = "name@company.com";  
var n = 0.00;  
var i = 0, j = 0, k;  
var isMouseOverLink = false;  
var helloMessage = "Hello, thank you for coming!";  
k = 0;
```

Нескладно помітити, що всі змінні, крім «`k`», були проініціалізовані під час оголошення, а «`k`» ініціалізується пізніше. JavaScript читає рядки коду зверху вниз, виконуючи директиви одну за одною. До ініціалізації змінна залишається невизначеною (`undefined`), тому прочитати її значення не можна. Спроба читання значення змінної перед її ініціалізацією призведе до помилки під час виконання програми. JavaScript дає можливість перевірити, чи було присвоєно значення змінної, за допомогою оператора **`typeof k`**.

На додаток до ключового слова `var`, JavaScript пропонує й інший спосіб оголошення змінних за допомогою звичайної їх ініціалізації. Якщо привласнити новій змінній значення перед її оголошенням за допомогою `var`, то JavaScript автоматично оголосить її.

Типи даних.

При збереженні порції даних (більш відомих під ім'ям значення), JavaScript автоматично відносить її до однієї з п'яти категорій. У табл. 2.2 зведені різні типи даних, підтримувані JavaScript.

Таблиця 2.2. Типи даних JavaScript.

Тип	приклад
number	-19,3.14159
boolean	true, false
string	"Елементарно, дорогий Ватсон", ""
function	unescape, write
object	window, document, null

Змінні типу `number` зберігають справжні чи цілі числа, `boolean` - значення `true` або `false`, а `string` містять рядкові літерали, включаючи порожній рядок. У табл. 2.2 порожній рядок представляється за допомогою двох подвійних лапок. Функціональний тип (`function`) або

визначається користувачем, або ставиться до вбудованих функцій. Наприклад, функція `unescape` є вбудованою в JavaScript.

Функції, що належать об'єктам, в JavaScript називаються методами (methods). Вони також відносяться до типу даних `function`. Базові клієнтські об'єкти JavaScript, такі як `window` або `document`, мають тип даних `object`. Змінні типу `object`, або просто об'єкти, можуть зберігати об'єкти. Мінлива зі значенням `null` відноситься до типу `object`. Ініціалізація змінної значенням `null` дозволяє уникнути помилок, якщо немає впевненості в майбутньому її використанні.

Як правило, в інших мовах програмування потрібно визначати тип даних, які подаються нової змінної. Протягом усієї програми тип будь-якого значення, що присвоюється змінної, повинен бути певним і незмінним. Більш того, якщо змінної присвоюються значення різних ТИПІВ, це призводить до помилки. Для JavaScript, слабо типизированного мови, подібне не має місця. Чи не потрібно визначати типи даних. Крім того, однією і тією ж змінною можна присвоювати значення різних типів. Мінлива JavaScript здатна в будь-який момент прийняти нове значення. Розглянемо приклади допустимого використання змінних JavaScript:

```
var carLength;  
carLength = 4 + 5;  
document.writeln(carLength);  
carLength = "9 футів";  
document.writeln(carLength);
```

Після оголошення змінної **carLength** їй присвоюється значення `4 + 5`. В JavaScript число `9` зберігається з типом `number`. Привласнюючи **carLength** значення іншого типу `"9 футів"`, в цій змінній зберігається строковий літерал (`string`). Операція подібного роду зменшує кількість дій, які могли б знадобитися в інших мовах програмування для повідомлення про факт зміни типів значень.

Область дії (Або видимості) змінних (`scope`) - це ділянки всередині програми, в яких можливе посилання на змінну. Наприклад, розглянемо випадок застосування єдиного сценарію в частині `<head>` HTML-документа, а іншого - в частину `<body>` того ж документа. JavaScript вважає, що всі змінні, оголошені всередині згаданих двох ділянок, належать одній і тій же області дії. Такі змінні є глобальними (`global`), і вони доступні будь-якому сценарію в поточному документі. Трохи пізніше мова піде про функції - окремих блоках коду. Змінні, оголошені всередині таких блоків, носять назву локальних (`local`) і вони доступні не кожному сценарію.

Локальні змінні

Змінна, оголошується всередині функції є локальною. До значення цієї змінної може мати доступ тільки дана функція. При кожному виклику функції всі локальні змінні створюються, а після повернення функції - руйнуються. Якщо в іншій функції присутній оголошення змінної з таким же ім'ям, вона розглядається як абсолютно інша змінна. Кожна зі змінних адресує свій блок пам'яті.

Глобальні змінні

Якщо потрібно, щоб якась змінна спільно використовувалася в більш ніж одній функції, слід оголосити її поза функцією (але, звичайно ж, всередині пари дескрипторів `<script>`). У цьому випадку дана змінна буде доступна в будь-якій частині всієї програми, в тому числі і у всіх функціях.

ПРИМІТКА

Рекомендується оголошувати глобальні змінні в розділі `<head>` HTML-сторінки, тим самим гарантуючи, що вони завантажуться раніше будь-якої іншої частини програми.

Лістинг 2.2 демонструє спосіб оголошення і застосування глобальної змінної. Для більшого розуміння відмінностей між областями дії змінних, в програму поміщені дві функції. Не обов'язково з'ясовувати, як діють функції, слід лише засвоїти той факт, що вони є окремими частинами сценарію, укладені у фігурні дужки (`{}`).

Лістинг 2.2. Глобальні і локальні змінні.

```
<!DOCTYPE html>
```

```

<html>
<head>
  <title>test</title>
  <script type="text/javascript ">
    // Глобальні змінні
    var globalString = "A";
    // Функції
    function changeToB () {
      document.outputForm.beforeB.value = globalString;
      globalString = "B";
      document.outputForm.afterB.value = globalString;
    }
    function changeToC () {
      document.outputForm.beforeC.value = globalString;
      globalString = "C";
      document.outputForm.afterC.value = globalString;
    }
  </script>
</head>
<body>
  <script type = "text/javascript ">
    document.write("Значення після ініціалізації змінної globalString одне
\" "+ globalString +\" \".");
  </script>
  <br>
  <form name = "outputForm">
    <input name = "changeButtonA" type ="button" value ="Змінити на B"
onclick ="changeToB()">
    <input name = "changeButtonB" type ="button" value ="Змінити на C"
onclick ="changeToC()">
    <br>
    Значення в globalString
    <input name = "beforeB" type ="text" size ="5,1"> <br>
    Після натискання на кнопку "Змінити на B"
    <input name = "afterB" type = "text" size = "5,1"> <br>
    Після натискання на кнопку "Змінити на C" <br>
    Поточне значення globalString
    <input name = "beforeC" type ="text" size ="5,1"> <br>
    після натискання
    <input name = "afterC" type ="text" size ="5,1"> <br>
  </form>
</body>
</html>

```

Першим відображається початкове значення globalString. Незважаючи на те що змінна оголошується в розділі <head> документа, сценарій, розташований всередині розділу <body> її може використовувати. Клацання на кнопці "Змінити на B" призводить до того, що функція changeToB() виводить на екран первісне значення globalString, замінює його на B і відображає нове значення globalString (див, рис. 2.2).

Значення після ініціалізації змінної `globalString` одне " A ".

Змінити на В	Змінити на С
Значення в <code>globalString</code> A	
Після натискання на кнопку "Змінити на В"	В
Після натискання на кнопку "Змінити на С"	
Поточне значення <code>globalString</code>	В
після натискання	С

МАЛЮНОК 2.2. Глобальні і локальні змінні.

Для демонстрації того, що всі функції в документі можуть використовувати одну і ту ж змінну, була додана друга кнопка для виклику іншої функції.

Після клацання на кнопці "Змінити на С", на екрані відображається поточне значення `globalString`, на даний момент дорівнює В, після чого значення `globalString` змінюється на С. На рис. 2.2 показані результати описаних дій.

Константи

Константа являє собою змінну, що зберігає своє значення незмінним на всьому протязі виконання програми. JavaScript використовує вбудовані константи для представлення значень для звичайних математичних дій, скажімо, число пі. Доступ до таких констант здійснюється через об'єкт `math`.

Константи, визначені користувачем - це суть певні користувачем змінні, значення яких не змінюються. Зазвичай при іменуванні констант використовуються малі літери і вони визначаються на початку програми.

JavaScript не підтримує константи в традиційному сенсі. У мовах програмування, що підтримують константи, які визначаються користувачем, ніяка інша частина програми не може змінювати значення константи після його визначення. Спроба подібного зміни призведе до помилки. В JavaScript все по-іншому.

Навіть при відсутності впевненості в тому, що змінна не змінюється, все ж варто працювати зі змінними для збереження значень, які будуть використовуватися протягом усього сценарію. Замінюючи змінної безліч входжень простих значень, сценарій набагато простіше змінити згодом. Все що потрібно зробити - змінити початкове значення змінної і сценарій повністю оновиться.

Кольори

JavaScript підтримує кольору, використовувані при створенні HTML-сторінок. Доступні кольори у вигляді строкових літералів (НЕ констант) зведені в табл. 5.8. Перераховані значення можна привласнювати певним властивостям об'єкта.

У лістингу 2.3 показано застосування цих значень для установки кольорів відображаються рядків. Допускається використання імені кольору або його шестнадцятиричного еквівалента. Хоч це може здатися дивним, проте виконується тільки установка властивості `fontcolor` для відображуваного на екрані строкового літерала. Непогано було б уважно вивчити рис. 2.3.

ПРИМІТКА

Не можна змінити колір тексту, вже виведеного у вікно браузера. У такому випадку потрібно просто перезавантажити документ з новим встановленим кольором шрифту.

Лістинг 2.3. Використання квітів в JavaScript.

```
<!DOCTYPE html>
<html>
<head>
  <title>test</title>
</head>
<body>
```

```

<h2>
  <script type="text/javascript ">
    document.write("Пікнік дня співробітників
компанії!".fontcolor("crimson"));
  </script>
</h2>
<h4>
  <script type="text/javascript ">
    document.write("19 липня о першій годині дня".fontcolor("blue"));
    document.writeln("<br>" + "Не забудьте захопити свою
сім'ю!".fontcolor("#008000"));
  </script>
</h4>
</body>
</html>

```

Шістнадцяткова рядок для кожного кольору є комбінацією значень RGB (red/green/blue - червоний / зелений / синій), що лежать в основі кожного кольору. Наприклад, шістнадцяткове значення для кольору морської хвилі (aqua) виглядає як # 00FFFF. У наведеному значенні RGB: 00 червоний, FF зелений і FF синій. Дві цифри в шістнадцятковому значенні дозволяють визначати 256 відтінків кожного кольору. В кольорі морської хвилі червона складова дорівнює 0, а це означає відсутність в ньому червоного кольору. Інтенсивності зеленого і синього максимальні. Забезпечивши відповідні пропорції зеленого і синього в результаті призводять до кольору морської хвилі. Маючи 256 комбінацій червоного, синього і зеленого можна скомпонувати близько 16 мільйонів квітів, однак не всі з них підтримуються JavaScript.

Пікнік дня співробітників компанії!

19 липня о першій годині дня

Не забудьте захопити свою сім'ю!

МАЛЮНОК 2.3. Зміна, вета тексту в JavaScript.

Типи даних

Незважаючи на недавнє обговорення основних типів даних, що привласнюються змінним, варто нагадати, що функції і об'єкти відносяться до спеціальних типів даних. Вони пропонують досить цікаві способи збереження і маніпулювання даними сценарію.

Вирази

Вираз (Expression) - це набір операторів, які подібно групі об'єднується в одне значення. Результуюче значення буде належати до одного з наступних типів даних: boolean, number, string, function, object.

Вираз може бути простим, як окреме число або змінна, а також включати в себе змінні, ключові слова і операції. Наприклад, вираз $x = 10$ привласнює значення 10 змінної x . Наведене вираз дає результат 10, тому застосування вирази в рядку а-ля `document.writeln (x = 10)` - цілком допустимо.

JavaScript міг би розглядати інформацію, укладену в лапки, як рядок і просто відображати її на екрані, проте в даному випадку виконуються деякі додаткові дії. Спочатку розраховуються всі вирази, присутні в дужках, і тільки після цього здійснюється власне відображення значення на екрані. У розглянутому прикладі на екран виводиться 10.

До речі, просто x - теж допустимий вираз. В даному випадку виконується тільки читання значення змінної з пам'яті. Чи не виконується жодного присвоювання

Коментарі

До сих пір дескриптори коментаря застосовувалися тільки для оточення тексту сценаріїв, що гарантувало відсутність їх відображення в старих браузерях. А як щодо скористатися

коментарем, щоб проігнорувати частина коду JavaScript? Виявляється, все сильно змахує на C / C ++ і Java:

```
// Однорядковий коментар
```

Дві косих риси (//) "приховують" текст аж до кінця поточного рядка. Допускаються і багаторядкові коментарі:

```
/* Чи бажаєте помістити в коментар кілька рядків? Так ось вам і спосіб! */
```

В обох типах коментарів зайві прогалини ігноруються, тому обидва наступних прикладу коректні

```
// Коментар
```

```
/* Багаторядковий коментар */
```

Ще одне застосування коментарів стосується налагодження сценаріїв. Для цілей тестування можна заховати частину свого коду, а після його налагодження з легкістю повернути, видаливши всього лише символи коментаря. На відміну від видалення частин сценарію, такі дії суттєво економлять час.

Контрольні питання.

1. **Інтеграція в HTML:** Як правильно вбудувати JavaScript-код у HTML-документ? Опишіть використання тегу `<script>` та його атрибута `type="text/javascript"`.
2. **Розміщення скриптів:** У чому різниця між розміщенням скрипту в секції `<head>` та в секції `<body>`? Як це впливає на момент виконання коду?
3. **Токени (Tokens):** Що таке токени в JavaScript? Назвіть основні типи токенів (ідентифікатори, ключові слова, літерали тощо).
4. **Ідентифікатори:** Які правила існують для створення імен змінних (ідентифікаторів)? Чи чутливий JavaScript до регістру (наприклад, `myVar` і `MyVar`)?
5. **Змінні:** Як оголошуються змінні за допомогою ключового слова `var`? Чи обов'язкова ініціалізація змінної при оголошенні?
6. **Типи даних:** Які основні типи даних підтримує JavaScript (`number`, `boolean`, `string`, `function`, `object`, `null`, `undefined`)?
7. **Літерали:** Наведіть приклади літералів різних типів (цілі числа, шістнадцяткові числа, числа з плаваючою комою, рядки, булеві значення).
8. **Область видимості (Scope):** Поясніть різницю між глобальними та локальними змінними. Де доступна змінна, оголошена всередині функції?
9. **Коментарі:** Як створюються однорядкові (//) та багаторядкові (/* ... */) коментарі в JavaScript?
10. **Спецсимволи в рядках:** Як використовувати зворотний слеш (\) для екранування спеціальних символів (наприклад, `\n` для нового рядка або `\"` для лапок)?
11. **Метод document.write:** Для чого використовуються методи `document.write()` та `document.writeln()`? У чому різниця між ними?
12. **Вирази (Expressions):** Що таке вираз у JavaScript і які типи значень він може повертати?
13. **Кольори:** Як можна керувати кольором тексту за допомогою методів рядкового об'єкта, таких як `.fontcolor()` (наприклад, `!".fontcolor("red")`)?